

Introducción a la Programación I

Clase 1: Hello World y Variables

Agenda

- ¿Qué necesitas para empezar?
- Tu primer programa: Hello World
- Crear y ejecutar un archivo `.py`
- ¿Qué pasó cuando lo ejecutaste?
- Comentarios en Python
- Variables: qué son y cómo usarlas
- Convenciones de nombres
- Tipos numéricos: enteros y flotantes
- Operaciones matemáticas y precedencia
- Casting entre tipos
- Números grandes y asignación múltiple

¿Qué necesitás para empezar?

Para escribir y ejecutar tu primer programa necesitás tres cosas:

1. El intérprete de Python 3

- Descargalo en python.org/downloads

2. Un editor de texto

- Cualquier editor sirve. Recomendado: **VS Code** o **Sublime Text**

3. Una terminal / consola

- En macOS/Linux: Terminal
- En Windows: PowerShell o CMD

Si ya tenés todo instalado, estamos listos para empezar.

Tu primer programa: Hello World

El programa tradicional de introducción a cualquier lenguaje.

- Solo hace **una cosa**: mostrar el mensaje `"Hello World!"` en pantalla
- Para crearlo vas a:
 - i. Crear un **archivo fuente** (`.py`)
 - ii. Escribir el código
 - iii. **Ejecutarlo** con el intérprete de Python

Crear el archivo: hello_world.py

1. Abrió tu editor de texto
2. Creá una carpeta llamada `prog1-python` en un lugar conocido de tu computadora
3. Guardá un archivo vacío con el nombre `hello_world.py`
4. Escribí lo siguiente:

```
# The following line will print a Hello World message to the screen  
print("Hello World!")
```

5. Guardá el archivo

El nombre del archivo usa `snake_case` — palabras en minúscula separadas por guión bajo.

Ejecutar el programa

1. Abrí tu terminal

2. Navegá hasta la carpeta `prog1-python` con el comando `cd` :

```
cd prog1-python
```

3. Ejecutá el programa:

```
python3 hello_world.py
```

4. Deberías ver en pantalla:

```
Hello World!
```

En Windows puede ser `python` en lugar de `python3` .

¿Qué pasó cuando ejecutaste hello_world.py?

```
# The following line will print a Hello World message to the screen  
print("Hello World!")
```

- El **intérprete de Python** leyó el archivo para determinar su contenido
- Vio la función **print** seguida de paréntesis
- Imprimió en pantalla lo que estaba **adentro** de los paréntesis: **"Hello World!"**
- La primera línea (con **#**) fue **ignorada** — es un comentario

Tu editor también hizo cosas:

- Resaltó diferentes partes con distintos colores → **syntax highlighting**
- **print()** aparece en un color, **"Hello World!"** en otro

Comentarios en Python

¿Qué fue esa primera línea?

```
# The following line will print a Hello World message to the screen
```

- El símbolo `#` indica un **comentario**
- Todo lo que sigue en esa línea es **ignorado por el intérprete**
- Es solo para vos (y otros programadores que lean el código)

¿Qué comentarios vale la pena escribir?

- Explicar **por qué** hace algo, no solo qué hace
- Ayudan cuando volvés al proyecto después de tiempo
- Facilitan que otros puedan contribuir a tu código

```
# Calculate the average of three scores
```

Variables — Modificando hello_world.py

Podemos guardar el mensaje en una **variable** y luego usarla:

```
# The following program will print a Hello World message to the screen  
message = "Hello World!"  
print(message)
```

¿Qué resultado produce este programa?

Variables — El mismo resultado

```
message = "Hello World!"  
print(message)
```

Produce exactamente lo mismo:

```
Hello World!
```

Pero ahora el mensaje está almacenado en `message` .

Podemos **cambiar** el valor de la variable y el programa se comportará diferente.

```
message = "Hello World!"  
print(message)  
message = "Hello Argentina!"  
print(message)
```

¿Qué es una variable?

Definición clásica: una caja donde guardás valores.

Útil para empezar, pero no del todo precisa.

Mejor definición: una **etiqueta** que referencia un valor en memoria.

message —————> "Hello World!"

- La variable `message` **apunta** al valor `"Hello World!"`
- Podés cambiar a qué valor apunta en cualquier momento
- Una variable **referencia** un cierto valor

En Python, las variables no tienen un tipo fijo — pueden apuntar a cualquier tipo de valor.

Convenciones para nombrar variables

Reglas que **toda** variable en Python debe respetar:

- Son **case sensitive**: `x` y `X` son variables distintas
- Pueden contener letras, números y guiones bajos `_`
 - Este estilo se llama **snake_case**
- **No** pueden comenzar con un número
 -  `message_1`  `1_message`
- **No** pueden tener espacios (usá `_` en su lugar)
 -  `greeting_message`  `greeting message`
- **No** uses palabras reservadas de Python (`if`, `for`, `print`, etc.)

Convenciones para nombrar variables (cont.)

Reglas de buenas prácticas:

- Nombres **cortos pero descriptivos**:
 -  `name`  `n`
 -  `student_name`  `s_n`
 -  `name_length`  `length_of_person_name_string`
- Evitá empezar con mayúsculas (eso es para clases)
- Nombrá de forma **consistente** en todo el programa

```
# Will the following program print the message correctly?  
message = "Hello World!"  
print(mesage)    # ← typo! mesage vs message
```

Python es case sensitive y preciso — un typo genera error.

Números

Los programas frecuentemente trabajan con datos numéricos.

Python divide los números en dos tipos principales:

Tipo	Descripción	Ejemplo
<code>int</code> (entero)	Números sin parte decimal	<code>42</code> , <code>-7</code> , <code>0</code>
<code>float</code> (flotante)	Números con punto decimal	<code>3.14</code> , <code>-0.5</code> , <code>2.0</code>

"Float" viene de *floating point* — el punto decimal puede estar en cualquier posición.

Crear una variable entera

Para crear una variable entera usás el operador de **asignación** `=`:

```
# Some samples
actual_speed = 100
low_speed = 10
average_speed = actual_speed - low_speed

local_score = 2
away_score = 1
```

El `=` en programación **no significa igualdad** — significa **asignar** el valor de la derecha a la variable de la izquierda.

Operaciones básicas con enteros

```
>>> 2 + 3      # suma
5
>>> 3 - 2      # resta
1
>>> 2 * 3      # multiplicación
6
>>> 3 / 2      # división
1.5
```

También podés combinar operaciones y usar paréntesis:

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

Los espacios alrededor de los operadores no afectan el resultado, pero hacen el

Más operaciones: potencia, división entera, módulo

```
>>> 2 ** 3      # potencia (2 elevado a 3)
8
>>> 7 // 3      # división entera (resultado sin decimales)
2
>>> 7 % 3       # módulo (resto de la división)
1
```

Resumen:

Operador	Nombre	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	5 - 3	2
*	Multiplicación	5 * 3	15
/	División	7 / 2	3.5

Reglas de precedencia — PEMDAS

Cuando hay múltiples operaciones, Python sigue el orden **PEMDAS**:

Paréntesis → **Exponentes** → **Multiplicación/División** → **Adición/Sustracción**

```
# 1° Paréntesis
```

```
>>> (5 + 1) / 3
```

```
2.0
```

```
>>> 5 + 1 / 3
```

```
5.333...
```

```
# 2° Exponentes
```

```
>>> 5 ** 2 - 2
```

```
23
```

```
>>> 2 * 5 ** 2
```

```
50
```

Reglas de precedencia — PEMDAS (cont.)

```
# 3° Multiplicación / División  
>>> 1 + 2 * 3  
7
```

```
# 4° Adición / Sustracción  
>>> 3 + 8 / 2  
7.0
```

Tip: ante la duda, usá paréntesis para dejar explícito el orden deseado.

Floats — Números con punto decimal

```
>>> 0.1 + 0.1
0.2
>>> 0.2 + 0.2
0.4
>>> 2 * 0.1
0.2
```

Pero a veces aparecen **sorpresas**:

```
>>> 0.2 + 0.1
0.30000000000000004
>>> 3 * 0.1
0.30000000000000004
```

Esto pasa en **todos** los lenguajes de programación. Se debe a cómo la computadora representa números de punto flotante en binario. Por ahora, ignora los decimales extra

no es un error de Python

Mezclar enteros y flotantes

- Dividir dos enteros **siempre produce un float**
- Mezclar entero y float en una operación **produce un float**

```
# División de enteros → float
```

```
>>> 4 / 2
```

```
2.0
```

```
# Mezcla de tipos
```

```
>>> 1 + 2.0
```

```
3.0
```

```
>>> 2 * 3.0
```

```
6.0
```

```
>>> 3.0 ** 2
```

```
9.0
```

Python "promueve" el resultado al tipo más general para no perder información.

Casting — Convertir entre tipos

A veces necesitás convertir un tipo en otro.

int() → convierte a entero (trunca los decimales, no redondea):

```
>>> int(5.6)
5
>>> int(3.9)
3
>>> int("42")      # también funciona con strings numéricos
42
```

float() → convierte a flotante:

```
>>> float(5)
5.0
>>> float("3.14")
3.14
```

Números grandes — Uso de guiones bajos

Cuando trabajás con números muy grandes, podés usar `_` para agrupar dígitos (como un separador visual):

```
>>> universe_age = 14_000_000_000  
  
>>> print(universe_age)  
14000000000
```

El guión bajo es solo visual — Python lo ignora al evaluar el número. No aparece al imprimirlo.

Asignación múltiple

Podés asignar valores a **múltiples variables en una sola línea**:

```
>>> x, y, z = 0, 0, 0
```

O asignar el **mismo valor** a varias variables a la vez:

```
>>> x = y = z = 0
```

Esto es útil para inicializar varias variables relacionadas:

```
# Initialize coordinates  
latitude, longitude = -34.6037, -58.3816
```

La asignación múltiple hace el código más conciso, pero no abuses de ella — la legibilidad es prioritaria.

Resumen de operadores

Asignación

x = 10

Aritméticos

suma = 5 + 3 # 8

resta = 5 - 3 # 2

producto = 5 * 3 # 15

cociente = 7 / 2 # 3.5

potencia = 2 ** 3 # 8

div_ent = 7 // 2 # 3

modulo = 7 % 2 # 1

Casting

entero = int(3.7) # 3

flotante = float(5) # 5.0

Resumen de la clase (1/2)

Concepto	Idea clave
<code>print()</code>	Muestra texto en pantalla
<code>#</code> comentario	Ignorado por el intérprete, para el programador
Variable	Etiqueta que referencia un valor en memoria
<code>snake_case</code>	Convención de nombres en Python
<code>int</code>	Número entero sin decimales

Resumen de la clase (2/2)

Concepto	Idea clave
<code>float</code>	Número con punto decimal
<code>=</code>	Operador de asignación (no igualdad)
<code>**</code> , <code>//</code> , <code>%</code>	Potencia, división entera, módulo
PEMDAS	Orden de precedencia de operaciones
<code>int()</code> , <code>float()</code>	Casting entre tipos numéricos

Próxima clase

Clase 2: Strings y sus operaciones

- Qué es un string (cadena de texto)
- Operaciones con strings: concatenación, slicing, métodos
- f-strings para formato de texto
- Entrada del usuario con `input()`

Practicá creando un archivo `variables.py` y probá todas las operaciones que vimos hoy.